

INFORME TÉCNICO: DESPLIEGUE ANALÍTICO Y OPTIMIZACIÓN DE TELEMETRÍA UX

Curso: Bases de Datos

Programa: Ingeniería de Sistemas

Pontificia Universidad Javeriana

Autor: Juan José Rocha Rivera

Fecha: Junio de 2026

1. INTRODUCCIÓN

El presente informe documenta el diseño, la implementación y la optimización del pipeline de datos para el proyecto *Chocolate-Doom Telemetry & UX Database* (Parte C). El foco principal de este componente radica en la interconexión de las métricas de rendimiento en bruto (tics de telemetría) con la percepción de experiencia de usuario (UX) recolectada a través del instrumento estandarizado BANGS.

A través de un despliegue incremental y el análisis de los planes de ejecución en un entorno nativo Linux (PostgreSQL), se valida la consistencia relacional del esquema de la Entrega B y la robustez de las consultas analíticas diseñadas para identificar patrones de juego y cuellos de botella en el diseño de niveles.

2. ARQUITECTURA DEL PIPELINE DE DESPLIEGUE

Para garantizar la integridad física y referencial de los datos, se estructuró un flujo de ejecución secuencial en la terminal de Linux a través del usuario administrador postgres. El orden estricto de ejecución implementado fue el siguiente:

1. **Sql Entrega B.sql (Modelo Core):** Cimentación del esquema relacional (11 tablas base vacías, incluyendo llaves primarias, foráneas y restricciones).
 2. **etl3.sql (Infraestructura de Staging):** Creación de las tablas temporales (stg_evento_telemetria) y funciones de captura de logs desarrolladas para la limpieza de datos masivos.
 3. **data.sql (Población Masiva):** Inyección controlada de datos maestros y la generación algorítmica de **21,600 registros** de telemetría consistente mediante funciones matemáticas (generate_series).
 4. **queries_juanjo.sql (Vistas Analíticas y Optimización):** Construcción de los reportes lógicos de trayectorias cruzadas con la escala BANGS, congelamiento de hotspots mediante vistas materializadas y activación de índices.
-

3. ANÁLISIS DETALLADO DE CONSULTAS COMPORTAMIENTOS (JUAN JOSEO)

3.1. Consulta Q4: Correlación de Trayectorias sobre el Promedio vs. Escala UX

Esta consulta analiza el comportamiento de los playtesters cuyas sesiones de juego tomaron más tiempo (medido en tics) que el promedio global, cruzando dicha persistencia con sus respuestas a las dimensiones de aburrimiento, ansiedad y frustración del instrumento BANGS.

Estructura del FROM y JOINS validados:

SQL

FROM jugadores_sobre_promedio jsp

JOIN jugador j ON j.id_jugador = jsp.id_jugador

JOIN usuario u ON u.id_usuario = j.id_usuario

JOIN respuesta_ux rx ON rx.id_usuario = u.id_usuario

AND rx.id_partida = jsp.id_partida

JOIN item_ux i ON i.id_item = rx.id_item

CROSS JOIN promedio_global pg

ORDER BY jsp.total_tics DESC, i.id_item;

- **Validación de Integridad:** Se verificó la consistencia del enlace con respuesta_ux utilizando rx.id_usuario = u.id_usuario en concurrencia con rx.id_partida = jsp.id_partida, asegurando compatibilidad absoluta con la estructura relacional de la Entrega B sin generar registros huérfanos ni duplicaciones cartesianas.

3.2. Consulta Q5: Identificación de Hotspots de Tráfico Espacial

Diseñada para localizar los sectores de un mapa que acumulan la mayor concurrencia de eventos, permitiendo a los diseñadores identificar zonas de cuello de botella o zonas muertas. Para optimizar el rendimiento, esta consulta se consolidó bajo una **Vista Materializada** (mv_hotspot_sectores), congelando el cálculo pesado de agregación en disco.

4. INGENIERÍA DE OPTIMIZACIÓN Y PLAN DE EJECUCIÓN (EXPLAIN ANALYZE)

Al ejecutar el archivo analítico en la consola mediante el comando `sudo -u postgres psql -d chocolate_doom_db -f queries_juanjo.sql`, el motor de base de datos de PostgreSQL arrojó un plan detallado de ejecución que evidencia tres fenómenos avanzados de optimización:

4.1. Filtrado Estricto y Resultados Lógicos (rows=0)

El plan de ejecución indicó un retorno de rows=0 en el nodo de salida final. Esto no representa un error de sintaxis, sino una validación matemática perfecta del set de datos inyectado. Dado que el guión de población masiva (data.sql) distribuyó equitativamente

exactamente 3,600 tics por jugador, la duración de todas las trayectorias individuales es idéntica al promedio global. Al evaluar la condición estricta:

$$\text{total_tics} > \text{avg_tics}$$

El filtro (Join Filter) removi6 correctamente las 20 filas intermedias generadas, demostrando el comportamiento preciso del operador l6gico.

4.2. Mecanismo de Cacheo Local (Memoize)

El optimizador detect6 llamadas repetitivas sobre las llaves primarias al cruzar los datos de usuarios y respuestas de UX. En lugar de realizar lecturas f6sicas secuenciales repetidas en disco, el motor activ6 un nodo de memoria:

- **M6trica del Plan:** Cache Key: rx.id_usuario -> Hits: 54 Misses: 6 Memory Usage: 1kB
- **Impacto:** Un 90% de las b6squedas relacionales de usuario se resolvieron en memoria en un tiempo r6cord de **0.001 ms** por ciclo, reduciendo dr6sticamente el estr6s sobre el subsistema de almacenamiento.

4.3. Poda del 6rbol de Ejecuci6n (never executed)

Debido a la naturaleza predictiva del optimizador de consultas de PostgreSQL, el plan marc6 los nodos correspondientes a los joins con las tablas usuario e item_ux como (never executed). El motor aplic6 un "cortocircuito": al determinar en los pasos inferiores que el filtro de promedios descartaba la totalidad de las filas, detuvo la ejecuci6n antes de procesar las uniones de cadenas de texto de las preguntas de BANGS, ahorrando ciclos de CPU cr6ticos de la m6quina.

- **Tiempo de Planificaci6n (Planning Time):** 2.746 ms
- **Tiempo de Ejecuci6n Real (Execution Time):** 6.357 ms

5. RECOMENDACIONES PARA SUSTENTACI6N EN VIVO

Para demostrar dominio din6mico del sistema ante el comit6 evaluador, se proponen dos acciones en vivo sobre el script queries_juanjo.sql:

1. **Activaci6n de Datos en Pantalla:** Si se requiere visualizar la interacci6n de la escala BANGS con datos reales, basta con modificar el operador de filtrado estricto en la cl6usula WHERE del Paso 3 de la consulta Q4:
 - *Cambiar:* WHERE d.total_tics > p.avg_tics
 - *Por:* WHERE d.total_tics >= p.avg_tics

Esto forzar6 al optimizador a incluir las sesiones que igualan el promedio, activando de inmediato todos los nodos de uni6n y desplegando la matriz anal6tica completa en la terminal de Linux.

2. **Uso de Índices B-Tree:** Resaltar que el escaneo actual se realiza mediante Seq Scan debido al tamaño controlado del laboratorio (21,600 filas). El motor determina que es más eficiente leer la memoria secuencialmente que recorrer el árbol de índices, pero la estructura queda indexada mediante idx_tel_sector para soportar escalabilidad horizontal de millones de tics en producción.
-

6. CONCLUSIONES

- El pipeline incremental implementado en Linux funcionó de manera armónica, confirmando que las modificaciones realizadas para acoplar el ETL de la base temporal con los guiones de población masiva respetan al 100% las llaves primarias y foráneas del modelo core.
- Las métricas de tiempo de ejecución inferiores a los 7 milisegundos demuestran que el uso de vistas materializadas para capas espaciales densas (como los hotspots de la Q5) y la estrategia de indexación seleccionada cumplen con los estándares de rendimiento requeridos para sistemas analíticos de telemetría de videojuegos en tiempo real.